Guia Completo para Apresentação: SARSA, Q-Learning e n-step SARSA

Ambiente: `CliffWalking-v0 / v1`
Disciplina: Aprendizado por Reforço
Laboratórios Abordados: Laboratório 7 (SARSA), Laboratório 8 (Q-Learning) e Laboratório 9 (n-step SARSA).

---

1. Visão Geral do Problema Comum: `CliffWalking`

O comportamento de todos os algoritmos decorre de como eles reagem à dinâmica e às penalidades deste ambiente.

1.1. Grade e Estrutura dos Estados

É um Gridworld de $4 \times 12 = 48$ estados (4 linhas $\times$ 12 colunas):

```
Linha 0: [ 0] [ 1] [ 2] [ 3] [ 4] [ 5] [ 6] [ 7] [ 8] [ 9] [10] [11]  (Topo - Rota Longa Segu
Linha 1: [12] [13] [14] [15] [16] [17] [18] [19] [20] [21] [22] [23]
Linha 2: [24] [25] [26] [27] [28] [29] [30] [31] [32] [33] [34] [35]  (Rente à Borda do Abism
Linha 3: [36] [37] [38] [39] [40] [41] [42] [43] [44] [45] [46] [47]  (Fundo)
         (S)  [------------------- PENHASCO ------------------]  (G)
```

- **Início (S):** Estado 36 (canto inferior esquerdo).
- **Alvo (G):** Estado 47 (canto inferior direito - terminal).
- **Penhasco:** Estados 37 a 46.
- **Espaço de Ações:** 4 ações discretas: `0` (Cima), `1` (Direita), `2` (Baixo), `3` (Esquerda).

1.2. Recompensas e Dinâmica

- **-1** para cada passo comum em terra firme.
- **-100** ao cair no penhasco, e o agente é imediatamente teleportado de volta ao início ($S=36$).
- **Limite de passos:** `MAX_EPISODE_STEPS = 500` (caso esgote, o episódio é truncado).

1.3. O Dilema Central da Tarefa

1. **Caminho Ótimo Estrito (Mínimo de passos):**
    - 13 passos rente ao penhasco (passando pela Linha 2): sobe 1, anda 11 para a direita, desce 1 no alvo.
    - Retorno total determinístico: **-13**.
    - **Risco:** Qualquer desvio exploratório para baixo derruba o agente no penhasco (penalidade -100).
2. **Caminho Seguro:**
    - 15 a 17 passos contornando pelas linhas superiores (Linhas 0 ou 1).
    - Retorno determinístico: **-15 a -17**.
    - **Vantagem:** Mesmo se uma ação exploratória for sorteada para baixo, o agente não cai no precipício.

---

2. Laboratório 7: SARSA (1-step On-Policy TD Control)

2.1. Teoria

O **SARSA** (*State-Action-Reward-State-Action*) é um algoritmo **on-policy**. Ele avalia e aprimora a **mesma política** que está sendo executada para gerar as ações no ambiente (a política comportamental ϵ-gulosa).

Equação de atualização: $$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \Big[\underbrace{R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})}_{\text{Alvo TD: } \bar{q}_t} - Q(S_t, A_t) \Big]$$

> **Ponto fundamental:** A ação A_{t+1} no alvo TD é a ação que o agente **realmente escolheu** segundo sua política ϵ-gulosa. O SARSA leva em consideração os desvios exploratórios que ele mesmo comete.

2.2. Como foi Implementado no Código

- **`update_policy(obs)`:** Atualiza a matriz `Pi[obs]` para a distribuição ϵ-gulosa:
    - Todas as ações recebem a probabilidade base: $\frac{\epsilon}{|\mathcal{A}|}$.
    - A ação gulosa ($\arg\max_a Q(s, a)$) recebe: $1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|}$.
- **`get_target_value(next_obs, next_action, reward, terminated)`:**

```
if terminated:
    target_value = reward
else:
    target_value = reward + self.discount_factor * self.Q[next_obs, next_action]
```

- **`update_Q(...)`:** Calcula o erro TD: $\delta_t = Q(s_t, a_t) - \text{target_value}$ e atualiza $Q(s_t, a_t) \leftarrow Q(s_t, a_t) - \alpha \cdot \delta_t$.
- **`run_episode_and_train(...)`:**
    1. No estado s_t, executa a_t via `env.step(a_t)`.
    2. Recebe $s_{t+1}, r_{t+1}, \text{terminated}$.
    3. **Sorteia antecipadamente a próxima ação** $a_{t+1} \sim \pi(\cdot|s_{t+1})$.
    4. Executa a atualização completa com $(s_t, a_t, s_{t+1}, a_{t+1}, r_{t+1})$.
    5. Atualiza as variáveis: $s_t \leftarrow s_{t+1}$ e $a_t \leftarrow a_{t+1}$.

2.3. Perguntas e Respostas Teóricas da Tarefa

Tarefa 0: Implementação

- **(a) Comparação da trajetória gulosa com o caminho mais curto (13 passos):**
  O SARSA não escolhe o caminho mais curto de 13 passos rente ao abismo; ele aprende uma rota mais longa e segura pelo topo (15 a 17 passos). Isso ocorre porque o SARSA é *on-policy*: seu alvo utiliza A_{t+1} amostrado da política ϵ-gulosa. Como há probabilidade ϵ de explorar aleatoriamente, andar na beirada tem alto risco de queda (penalidade -100). Esse risco é incorporado aos valores Q da borda, fazendo o agente preferir contornar pelo alto.
- **(b) Recompensa da trajetória gulosa (-15) vs Avaliação ϵ-gulosa (-21.33):**
  A trajetória gulosa é avaliada de forma determinística ($\epsilon=0$), executando sempre a melhor ação sem desvios. Já a avaliação com `evaluate_policy` utiliza a política ϵ-gulosa, onde o agente ainda realiza ações exploratórias que prolongam o caminho e reduzem o retorno médio.

Tarefa 1: Efeito de α (Taxa de Aprendizado)

- $\alpha = 0.01$: Aprendizado lento demais. Em 1000 episódios, o agente não propaga os valores a tempo até a origem.
- $\alpha = 0.1$: Equilíbrio adequado entre estabilidade e velocidade de convergência.

- $\alpha = 1.0$: O agente substitui completamente o valor antigo pelo novo alvo ($Q \leftarrow \text{alvo}$). Devido ao ruído da exploração ϵ-gulosa, o aprendizado se torna muito oscilante e instável.
- As trajetórias gulosas podem divergir entre execuções devido à estocasticidade das ações exploratórias durante o treino e ao critério de desempate no $\arg\max$.

Tarefa 2: Efeito de γ (Fator de Desconto)

- $\gamma = 0.0$ e $\gamma = 0.5$: O agente apresenta **miopia temporal**. Como todo passo no grid dá recompensa -1, e o desconto é nulo ou reduz excessivamente o retorno futuro, o agente não percebe vantagem em alcançar o objetivo. Todas as ações parecem igualmente ruins (empate em -1), fazendo o agente ficar preso e atingir o limite de 500 passos (truncamento).
- $\gamma = 0.9$ e $\gamma = 0.99$: O agente valoriza recompensas futuras a longo prazo e aprende a rota até o alvo.

Tarefa 3: Efeito de ϵ (Taxa de Exploração)

- À medida que ϵ aumenta ($0.001 \to 0.3$), o SARSA se afasta cada vez mais do penhasco (sobe para as linhas 1 e 0). Quanto maior o ruído exploratório, maior o risco de cair se estiver perto da borda.
- A diferença entre a recompensa da trajetória gulosa e da política aprendida aumenta com ϵ, pois maiores valores de ϵ introduzem mais desvios durante a avaliação.

Tarefa 4: Número de Episódios (N)

- Com $N=1, 10, 100$, o agente não tem episódios suficientes para que o sinal TD retropropague do alvo até o estado inicial. Somente com $N=1000$ o agente converge de forma consistente para o objetivo.

---

3. Laboratório 8: Q-Learning (1-step Off-Policy TD Control)

3.1. Teoria

O **Q-Learning** é um algoritmo **off-policy**. Ele estima diretamente a função valor da **política ótima** q_* (usando uma política-alvo puramente gulosa), enquanto coleta dados no ambiente por meio de uma **política comportamental exploratória** (ϵ-gulosa).

Equação de atualização: $$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \Big[\underbrace{R_{t+1} + \gamma \max_{a} Q(S_{t+1}, a)}_{\text{Alvo TD: } \bar{q}_t} - Q(S_t, A_t) \Big]$$

> **Ponto fundamental:** O operador $\max_a Q(S_{t+1}, a)$ assume que, a partir do próximo estado, o agente **sempre escolherá a ação ideal**, ignorando se a política que está interagindo com o ambiente é ϵ-gulosa e passível de cometer erros.

3.2. Como foi Implementado no Código

- **`get_target_value(next_obs, reward, terminated):`**

  ```
  if terminated:
      target_value = reward
  else:
      target_value = reward + self.discount_factor * np.max(self.Q[next_obs])
  ```

  Diferença para o SARSA: Não depende de `next_action`. Utiliza diretamente `np.max(...)`.

- **`run_episode_and_train(...):`** Não é necessário sortear a próxima ação antes de atualizar Q. Executa a_t, observa s_{t+1}, atualiza $Q(s_t, a_t)$ com base no $\max_a Q(s_{t+1}, a)$, e só então sorteia a ação seguinte no próximo passo.

3.3. Perguntas e Respostas Teóricas da Tarefa

Tarefa 0: Implementação

- **(a) Trajetória gulosa aprendida:**
  Coincide exatamente com o **caminho ótimo estrito de 13 passos** rente ao abismo (retorno -13). O operador $\max_a$ desconsidera o risco de tropeçar no penhasco em passos futuros, estimando diretamente a política ótima teórica.
- **(b) Trajetória gulosa (-13) vs Avaliação ϵ-gulosa (-50 a -70):**
  - Trajetória gulosa ($\epsilon=0$): O agente é puramente determinístico, anda colado à borda com precisão e nunca cai.
  - Avaliação ϵ-gulosa: Como a rota ótima passa encostada no abismo, qualquer passo aleatório sorteado na direção sul gera uma queda catastrófica (penalidade -100).

Tarefas 1, 2, 3 e 4: Hiperparâmetros

- Para todos os valores de ϵ, a trajetória gulosa converge para os mesmos 13 passos ótimos (propriedade *off-policy* de desacoplamento entre política-alvo e política de comportamento).
- Porém, na avaliação online com ϵ, valores maiores de ϵ provocam quedas frequentes e derrubam severamente o retorno médio.
- Para $N=1$ e $N=10$, o agente não consegue chegar ao alvo. Precisa de $N=100$ e $N=1000$ para propagar o retorno até a origem.

Tarefa 5: Comparação Completa: SARSA vs Q-Learning

- **Com $\epsilon = 0.1$:**
  - **Desempenho em Treinamento e Online (`evaluate_policy`):** O **SARSA vence** com folga (retorno médio entre -20 e -30, enquanto o Q-Learning fica entre -50 e -70). O SARSA aprendeu a rota conservadora que evita quedas durante a exploração.
  - **Política Gulosa Final (Deploy sem exploração):** O **Q-Learning vence** (13 passos, -13, contra 15 passos, -15 do SARSA).
- **O que acontece à medida que $\epsilon \to 0$:** Conforme $\epsilon \to 0$, a probabilidade de ações aleatórias tende a zero. O SARSA deixa de correr risco de quedas acidentais e passa a convergir exatamente para a mesma rota ótima rente ao abismo de 13 passos do Q-Learning. Os dois algoritmos tornam-se equivalentes.

---

4. Laboratório 9: n-step SARSA (Multi-step Bootstrap)

4.1. Teoria

O **n-step SARSA** unifica o TD de 1 passo e o Método de Monte Carlo:

- **1-step SARSA ($n=1$):** Atualização a cada passo com base em 1 recompensa imediata e bootstrap no estado seguinte. Baixa variância, mas alto viés inicial e propagação lenta do sinal de recompensa.
- **Monte Carlo ($n = \infty$):** Espera o episódio terminar e usa o retorno completo. Não usa bootstrap (viés zero), mas tem alta variância devido ao acúmulo de muitas ações estocásticas.
- **n-step SARSA:** Acumula **n recompensas reais descontadas** antes de truncar e aplicar o bootstrap com a estimativa $Q(S_{t+n}, A_{t+n})$:

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n Q(S_{t+n}, A_{t+n})$$

A atualização do par de estados e ações (S_τ, A_τ) ocorre com um **atraso de n passos**: $$\tau = t - n + 1$$ $$Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha \Big[G_{\tau:\tau+n} - Q(S_\tau, A_\tau) \Big]$$

4.2. Como foi Implementado no Código

- **Estruturas de Dados:** Listas/buffers de histórico: `states`, `actions`, `rewards` (com $R_0 = 0$).
- **`get_target_value(states, actions, rewards, tau, T_end, terminated)`:**

```
h = int(min(tau + self.n_steps, T_end))
target_value = 0.0
# 1. Soma das recompensas reais observadas no intervalo
for i in range(tau + 1, h + 1):
    target_value += (self.discount_factor ** (i - tau - 1)) * rewards[i]
# 2. Bootstrap com Q caso o horizonte h não seja terminal
if not (h == T_end and terminated):
    target_value += (self.discount_factor ** (h - tau)) * self.Q[states[h], actions[h]]
return target_value
```

- **Laço `while tau < T_end - 1:`:** Mesmo após o término do episódio no ambiente ($t \ge T_{end}$), o laço continua até que τ atinja $T_{end} - 1$.

4.3. Perguntas e Respostas Teóricas da Tarefa

Tarefa 0: Implementação

- **Por que o laço continua depois do último `env.step`?**
  Porque a atualização é defasada em n passos ($\tau = t - n + 1$). Quando o episódio termina no instante T, os últimos $n - 1$ pares visitados ainda estão pendentes de atualização no buffer. O laço precisa rodar por mais $n - 1$ iterações sem interagir com o ambiente para processar os alvos desses pares pendentes.
- **Com $n=4$, quais pares são atualizados e o que entra no alvo?**
  Ocorrem 3 iterações pós-término para atualizar:
  - Para $\tau = T - 3$: $\bar{q}_{T-3}^{(4)} = R_{T-2} + \gamma R_{T-1} + \gamma^2 R_T$
  - Para $\tau = T - 2$: $\bar{q}_{T-2}^{(4)} = R_{T-1} + \gamma R_T$
  - Para $\tau = T - 1$: $\bar{q}_{T-1}^{(4)} = R_T$
    Como o horizonte atinge o estado terminal ($h = T$), não há termo de

bootstrap ($Q = 0$). Os alvos tornam-se somas puras de recompensas reais observadas.

Tarefa 1: Efeito do Número de Passos ($n \in {1, 2, 4, 8, 16}$)

- **$n = 1$:** Propaga o reforço devagar, gerando mais episódios truncados no início.
- **$n \in {2, 4, 8}$:** Desempenho superior. Consegue encadear o efeito das quedas e da chegada ao alvo mais rapidamente até o início, acelerando a convergência.
- **$n = 16$:** Como o episódio no CliffWalking dura em média 15 a 17 passos, $n=16$ faz com que quase todas as atualizações sejam feitas pelo retorno real completo, sem bootstrap. Isso reduz o viés inicial de Q, mas eleva a variância em decorrência das escolhas aleatórias ao longo de muitos passos.
- **Fração sem bootstrap:** Para uma duração média de $L \approx 17$ passos, os últimos n passos não usam bootstrap ($\frac{\min(n, L)}{L}$):
  - $n=1: \approx 5.9%$
  - $n=2: \approx 11.8%$
  - $n=4: \approx 23.5%$
  - $n=8: \approx 47.1%$
  - $n=16: \approx 94.1%$
- **Aproximação quando $n \to \infty$:** Quando $n \ge L$, elimina-se o termo de bootstrap para todos os passos e o método torna-se idêntico ao **Método de Monte Carlo (MC)** on-policy.

---

5. Tabela Comparativa Resumida

Critério	SARSA (Lab 7)	Q-Learning (Lab 8)	n-step SARSA (Lab 9)
Tipo de Controle	On-policy	Off-policy	On-policy multi-passos
Fórmula do Alvo TD	$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$	$R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$	$\sum_{i=1}^n \gamma^{i-1} R_{t+i} + \gamma^n Q(S_{t+n}, A_{t+n})$
Ação usada no Alvo	Ação real amostrada da política ϵ-gulosa	Ação gulosa ideal $\arg\max_a Q$	Ação real n passos à frente

Critério	SARSA (Lab 7)	Q-Learning (Lab 8)	n-step SARSA (Lab 9)
Trajetória Gulosa	Rota Segura (15-17 passos, contorna pelo topo)	Rota Ótima Estrita (13 passos, rente ao penhasco)	Rota Segura (contorna pelo topo)
Retorno Médio no Treino	Alto (-20 a -30)	Baixo (-50 a -70, devido a quedas frequentes)	Alto e com convergência acelerada para n moderado
Tempo de Atualização	Imediato (a cada passo)	Imediato (a cada passo)	Atrasado em n passos ($\tau = t - n + 1$)
Comportamento quando $\epsilon \to 0$	Converge para a rota de 13 passos do Q-Learning	Mantém a rota de 13 passos	Converge para a rota de 13 passos do Q-Learning

---

6. Roteiro Sugerido para Apresentação Oral (5 Minutos)

1. **Introdução ao Problema (45 seg):**
    - Apresente o `CliffWalking` e explique a dinâmica: -1 por passo e penalidade severa de -100 ao cair no penhasco.
    - Enfatize o contraste entre a rota ótima de 13 passos (na beirada) e a rota segura de 15 passos (por cima).
2. **SARSA - On-Policy (1 min):**
    - Destaque que o alvo usa A_{t+1} real da política ϵ-gulosa.
    - Conclusão prática: Como o algoritmo antecipa que o agente pode errar e cair durante a exploração, ele aprende a rota segura pelo topo.
3. **Q-Learning - Off-Policy (1 min):**
    - Destaque que o alvo usa $\max_a Q$, assumindo comportamento perfeito no futuro.
    - Conclusão prática: Aprende a rota ótima estrita de 13 passos, mas sofre penalidades frequentes durante o treinamento devido à exploração rente ao penhasco.

4. **Comparação SARSA vs Q-Learning (1 min):**
    - Resuma o dilema: SARSA é melhor para o aprendizado online (maior segurança); Q-Learning encontra a política ótima teórica para execução determinística offline.
    - Explique que quando $\epsilon \to 0$, o medo de cair desaparece e ambos convergem para a mesma rota.
5. **n-step SARSA - Ponte entre TD e Monte Carlo (1 min 15 seg):**
    - Explique que o alvo acumula n passos reais antes de fazer bootstrap.
    - Explique por que o laço continua após o término do episódio (para esgotar o buffer de $n-1$ transições pendentes).
    - Mostre que valores intermediários de n ($2, 4, 8$) aceleram a propagação do sinal de recompensa e reduzem truncamentos.